

Smart HSRP Monitoring System: A Temporal Fusion Framework for Reliable Automated Traffic Violation Detection

Kshitij Sidram Lalde
Department of Computer Engineering
Trinity College Of Engineering and
Research Pune, India
kshitijlalde@gmail.com

Ketan Madhukar Butte
Department of Computer Engineering
Trinity College of Engineering and
Research Pune, India
ketanbutte17@gmail.com

Aniket Bijay Kumar Behera
Department of Computer
Engineering
Trinity College Of Engineering
and Research Pune, India
aniket.behera.0301@gmail.com

Prof. Mrs. Jareena Shaikh
Department of Computer
Engineering
Trinity College Of Engineering
and Research Pune, India
shaikhjareena.tcoer@kjei.edu.in

Parth Dattatray Date
Department of Computer Engineering
Trinity College Of Engineering and
Research Pune, India
parthdate2004@gmail.com

ABSTRACT

Automated traffic violation enforcement systems in India primarily use Automatic Number Plate Recognition (ANPR) and single-frame rule-triggered analytics. These systems often need human checks due to high false-positive rates. This paper presents the Smart High Security Registration Plate (HSRP) Monitoring System. It is a complete deep learning framework that addresses these issues through four main contributions: (1) a Temporal Decision Fusion engine that combines evidence from each frame over time, using Exponential Moving Average (EMA) scoring and adaptive bias correction to greatly reduce false detections; (2) a semi-supervised Adaptive Threshold Learner that continuously adjusts decision boundaries based on changes in the environment; (3) a composite Database Gating mechanism that applies a multi-factor quality score before making decisions; and (4) a proactive Violation Prediction engine that sends pre-enforcement warnings up to 20 frames before a confirmed event. Built on YOLOv8, YOLOv10, YOLOv11, EfficientNet-B0, DeepSORT, and EasyOCR, the system achieves 87.5% accuracy in HSRP classification, 97.9% plate detection mAP50, and 89.6% rider-class mAP50. Evaluations show that temporal fusion cuts single-frame false positives by over 60% while maintaining recall levels in line with frame-level baselines. The proposed architecture provides a scalable, hardware-independent solution that can integrate with existing city-level surveillance systems.

Keywords—HSRP verification, traffic violation detection, temporal decision fusion, adaptive threshold learning, YOLOv8, EfficientNet, DeepSORT, EasyOCR, intelligent transportation systems, false-positive reduction.

I. INTRODUCTION

In 2022, India saw over 461,000 road accidents and nearly 154,000 deaths—numbers straight from the Ministry of Road Transport and Highways. A big chunk of these cases comes down to people skipping helmets and vehicles running around with fake or non-HSRP license plates. These plates are tough to track, and they're often tied to stolen or uninsured vehicles. Even though the government made HSRP mandatory for all vehicles under Rule 50 of the Central Motor Vehicles Rules, 1989 (amended in 2019), enforcement is still patchy, especially in cities and their outskirts.

Right now, most of the enforcement tech in India—stuff from vendors like EFKON India and AIRPIX—relies on automatic number plate recognition (ANPR). These systems snap a photo, use optical character recognition (OCR) to read the plate, and check it against the national vehicle database. They work pretty well for things like speed checks and red-light violations, but when it comes to catching helmet violations or spotting fake plates, ANPR only scratches the surface. Human operators still have to step in before anyone gets a ticket because AI alone can mess up—false positives don't hold up in court.

Lately, researchers have been trying to take things further with deep learning for two-wheeler violations. UrbanEye, for example, uses a YOLO + OCR setup to spot helmetless riders, triple riding, and license plate issues. UrbanFlow brings in a microservice-based traffic management system with adaptive controls and ANPR. Both of these improve on what's out there, but they still focus on single frames—basically, they can't collect evidence over time. So, they run into the same problems as commercial systems: motion blur, blocked views, and changing light between frames.

But there's a way forward. Temporal aggregation—basically, looking at a series of frames instead of just one—has been a

hot topic in action recognition research. Fang and others have shown that blending predictions from different moments boosts accuracy in video tasks. The same logic works for traffic violations: single frames might leave you guessing, but patterns get clearer when you look at a sequence.

That's where the Smart HSRP Monitoring System comes in. It pulls together all these ideas into a real-world enforcement setup. The main features are: (i) a Temporal Decision Fusion (TDF) engine that gathers evidence using exponential moving average (EMA) scoring, adaptive bias correction, and a stability metric; (ii) a semi-supervised Adaptive Threshold Learner (ATL) that automatically tunes decision limits without needing labeled data; (iii) a Database Gating (DBG) mechanism to make sure only quality scores confirm violations; (iv) a Violation Prediction Engine (VPE) that can send alerts up to 20 frames in advance; and (v) a 7-layer frame processing pipeline that brings together detection, tracking, classification, OCR, and decision fusion in one package.

Here's how the rest of the paper goes: Section II looks at related research. Section III breaks down the system architecture. Section IV gets into the math behind each module. Section V covers memory management. Section VI shows the experimental results. Section VII lines up this system against what's already out there, and Section VIII wraps up with some thoughts on where to go next.

II. Related Work

A. Automated Traffic Enforcement Systems

Intelligent transportation systems are leaning more and more on computer vision and machine learning to spot traffic violations automatically. In the early days, people used radar triggers hooked up to magnetic induction loops and fixed cameras. Things really changed when GPU-powered deep neural networks came along. Suddenly, automatic number plate recognition (ANPR) got crazy accurate—over 95% in controlled environments. That kind of performance pushed cities like Bengaluru, Delhi, and Nagpur to roll these systems out everywhere.

Still, most of these setups focus on reading license plates. AI takes care of slicing up the plate image and recognizing the characters, while rule-based logic handles the rest—like flagging cars that go over the speed limit.

In India, companies like EFKON and AIRPIX are leading the way with their electronic enforcement and video analytics tools. Their systems can catch everything from red light running to speeding to people riding without helmets. But here's the thing: after the system flags a possible violation, the footage gets sent to a human team in a control room. They have to double-check everything before police take any action. So, even with all this tech, people still don't trust machine learning enough to let it make legal decisions on its own.

B. Deep Learning for Violation Detection

Govroju and his team built a system that spots multiple types of traffic violations—no helmets, wrong lanes, three people on a bike, and license plate issues. They use YOLOv8 to detect the violations and EasyOCR to read plates. On their

test dataset, the system picks up violations pretty well, but it doesn't keep track of things over time from frame to frame.

Patil's group took a different approach. They broke their system into microservices: one for license plate recognition, one for catching violations, and another for adaptive control. Everything comes together in a live dashboard, which makes it pretty appealing for cities. Still, their violation checks only look at single frames, and they don't share how often the system gets things wrong in real-world tests.

Fang's work goes in another direction, looking at how combining information from different parts of a video helps with action recognition. Instead of betting everything on the most obvious frame, they fuse clues from several segments and get better results. That idea actually inspired the EMA-based state model in our TDF engine, which now handles everything from action detection to enforcing violations.

C. Object Tracking and OCR

DeepSORT takes SORT a step further by adding a deep appearance descriptor. This means it can re-identify objects, even if they disappear for a moment or get blocked from view. Keeping the same track IDs matters a lot for piecing together information over time—you can still connect the dots when a car ducks out of sight for a bit. EasyOCR, on the other hand, uses a ResNet, BiLSTM, and CTC decoder to read non-standard Indian license plates, no matter how tricky the lighting gets.

This project doesn't just repeat what's been done—it pushes things forward. It adds a real temporal evidence layer, brings in a smart learning threshold, and uses prediction-aware database gating. None of these features showed up in earlier work.

III. SYSTEM ARCHITECTURE

The Smart HSRP monitoring system has a layered, modular setup with three main components: a backend processing engine (FastAPI + Uvicorn), an AI inference layer, and a frontend dashboard (Streamlit). Fig. 1 illustrates the entire data flow.

A. Backend Processing Engine

Users upload video feeds straight to the FastAPI REST endpoint. Once a video comes in, a background thread fires up `process_video()`. That's where everything gets set up: `FramePipeline`, `TemporalDecisionFusion`, `ViolationPredictor`, `DatabaseGate`, and `AdaptiveThresholdLearner` all kick in. For writing out processed integrals, there's an `AsyncVideoWriter` working off an in-memory queue that holds up to 200 integrals. It just writes them out—no attention to inference time or I/O speed.

The API gives you endpoints to upload videos, check on job status (`/process`, `/status/{job_id}`), get and process violations, look at adaptive thresholds (`/thresholds`), or even stream live annotated video. As for job statuses, those go in an in-

memory dictionary called JOB_STORE. Each job uses a UUID as its key.

B. 7-Layer Automated Processing Pipeline

The automated processing pipeline consists of seven steps for each video:

Layer 1 - Vehicle Detection: YOLOv11m can identify people, cars, motorcycles, gearboxes, and trucks with a confidence level of 0.4 or higher.

Layer 2 - Entity Splitting: Detections are combined into a stream of people and vehicles.

Level 3 - Object Tracking: DeepSORT uses the same track ID structure, allowing all systems to automatically maintain state.

Layer 4 - Plate Detection: YOLOv10s detects license plates on tracked vehicles (mAP50 = 0.979).

Layer 5 - HSRP Classification: EfficientNet-B0 classifies HSRP and NON_HSRP, and the results are sent to the TDF engine.

The Smart HSRP monitoring system has a layered, modular setup with three main components: a backend processing engine (FastAPI + Uvicorn), an AI inference layer, and a frontend dashboard (Streamlit). Fig. 1 illustrates the entire data flow.

C. Database Schema

The PostgreSQL schema uses four tables. The main one, violations, holds 19 fields for every event. You'll find things like track_id, vehicle_number, vehicle_class, and violin_type in there, along with violin_confidence, stability_score, temporal_consistency, consistent_detections, quality_score, and several flags—needs_manual_review, prediction_preceded. There's also some timing data: first_frame, last_frame, and track_duration_frames.

Then there's the users table. This one handles role-based access control, keeping credentials hashed with bcrypt for security. The adaptive_thresholds table is a bit different—it stores a row for each design type (HSRP, HELMET, and OCR), and the ATL module updates these as needed. We've also kept the legacy events and vehicle tables, so the schema still works with older ANPR systems.

You'll see users and events connect in a one-to-many relationship. That lets us track every enforcement action back to the operator session, which is key for chain of custody.

IV. MATHEMATICAL MODELS

A. Temporal Decision Fusion (TDF)

The TDF engine maintains a DecisionState for each (track_id, decision_type) pair. The decision_type can be one of {HSRP, HELMET, OCR}. Let $s_v(t)$ denote the raw model score for decision type v at frame t .

1) EMA Score

The smoothed score is calculated as:

$$\text{EMA}_v(t) = \alpha \cdot s_v(t) + (1 - \alpha) \cdot \text{EMA}_v(t-1)$$

Here, α is set to 0.20 for all decision types. This low α value ensures strong temporal inertia, making the fused score less responsive to isolated noisy detections. During frame skips, EMA decreases by a skip_decay factor of 0.96 for each skipped frame.

2) Adaptive Bias Correction

The bias term $B_v(t)$ is adjusted based on detection consistency. If consecutive_positive reaches 7 or more, the system adds a negative bias to make detection harder, countering model drift in situations with consistently high raw scores. When consecutive_negative is 5 or more, a positive bias lowers the effective threshold:

$$B_v(t) = \text{clip}(B_v(t-1) + \delta_v(t), -\beta_v, +\beta_v)$$

Here, β_v is the bias cap ($\beta_{\text{HSRP}} = 0.05$; $\beta_{\text{HELMET}} = \beta_{\text{OCR}} = 0.04$), and $\delta_v(t)$ is a signed increment based on the difference between $\text{EMA}_v(t)$ and the base threshold. The effective decision threshold is:

$$T'_v(t) = \text{clip}(T_v(\text{base}) + B_v(t), 0.10, 0.90)$$

where $T_{\text{HSRP}}(\text{base}) = 0.50$, $T_{\text{HELMET}}(\text{base}) = 0.40$, $T_{\text{OCR}}(\text{base}) = 0.45$.

3) Stability Metric

The stability score $\sigma_v(t)$ ranges from [0, 1] and measures decision consistency over the last $K = 5$ observations. Let $H_v(t) = \{s_v(t), s_v(t-1), \dots, s_v(t-K+1)\}$ represent recent history, maintained as a deque of length 15 for the entire TDF history:

$$\sigma_v(t) = 1 - \text{Var}(H_v(t)) + \text{bonus_streak}$$

where bonus_streak equals 0.05 for each consecutive frame beyond a set streak threshold. High stability indicates that the model consistently observed the same class label, which supports enforcement action.

4) Violation Confirmation Criteria

A violation for type v is confirmed only when all three conditions occur simultaneously:

$$\text{frames}_v \geq \text{min_frames}_v \text{ AND } \text{avg}(\text{EMA}_v) \geq \text{conf}_v \text{ AND } \text{avg}(\sigma_v) \geq \text{stab}_v$$

where $\text{min_frames}_{\text{HSRP}} = 3$, $\text{conf}_{\text{HSRP}} = 0.70$, $\text{stab}_{\text{HSRP}} = 0.65$; $\text{min_frames}_{\text{HELMET}} = 3$, $\text{conf}_{\text{HELMET}} = 0.50$, $\text{stab}_{\text{HELMET}} = 0.65$, $\text{mean_stab}_{\text{HELMET}} = 0.55$. The lower confidence requirement for helmet violations indicates a deliberate

policy bias toward recall. Missing a rider without a helmet is viewed as more costly than a false positive.

B. Adaptive Threshold Learning (ATL)

The ATL module implements online semi-supervised learning. Predictions with a confidence level of 0.75 or above are classified as pseudo-labeled ground truth. The online performance estimators (true positives, false positives, true negatives, and false negatives) are updated after each pseudo-labeled sample. The threshold is updated every $N = 500$ samples using a momentum-corrected rule:

$T_v \rightarrow T_v + \eta \cdot (\text{precision}_v - \text{target_precision}) \cdot \text{momentum}$, where η represents the learning rate and target_precision is set at 0.90. Threshold states are recorded as `state/thresholds.json` after each batch update, enabling for improvement across deployment sessions without the need for manually labeled data.

C. Database Gating Quality Score

Before inserting a violation record into PostgreSQL, the DBG module calculates a composite quality score Q within the range $[0, 1]$.

$$Q = 0.35 \cdot \bar{C} + 0.25 \cdot \bar{\sigma} + 0.25 \cdot \bar{\tau} + 0.15 \cdot n_{\text{norm}},$$

where $\bar{C}$ is the mean confirmation confidence over the last 10 observations, $\bar{\sigma}$ is the mean stability score, $\bar{\tau}$ represents mean temporal consistency (the proportion of frames in the window where the same label was predicted), and $n_{\text{norm}} = \min(1.0, \text{consecutive} / (\text{min_consecutive}))$. A violation record is kept only when Q is 0.55 or higher. Records with $Q < 0.55$ are rejected. Records with a Q of 0.55 or above but no prior VPE prediction (`prediction_preceded = False`) are saved but marked with `needs_manual_review = True`, resulting in a tiered confidence output.

D. Violation Prediction Engine (VPE)

The VPE sends out pre-violation alerts up to $L = 20$ frames before the confirmed event. Score histories of length 25 are stored separately for HSRP and helmet selection. A linear regression over the latest 5 samples yields a slope estimate, $\hat{m}$. The risk score R is calculated as follows:

$$R(t) = (1 - w_t) \cdot \text{EMA}_v(t) + w_t \cdot (\text{EMA}_v(t) + \hat{m} \cdot L) + P_{\text{bonus}},$$

where $w_t = 0.40$ is the trend weight, $L = 10$ is the lookahead frame count, and $P_{\text{bonus}} = 0.10$ for rising or persistent high trend patterns. The system transitions from NORMAL to WARNING ($R \geq 0.55$), then CONFIRMED ($R \geq 0.70$), and finally CLEARED. The annotation colors vary as follows: grey for tracking only, yellow for WARNING, red for CONFIRMED, and green for compliance.

V. MEMORY MANAGEMENT

The system processes hour-long video feeds in production. Without limits on memory growth, it would be unusable. The following strategies ensure limited memory usage.

A. Bounded In-Memory Structures

Each track keeps its own history in a Python deque, set to a maximum size. For TDF `DecisionState`, that history only stores the last 15 observations. VPE `TrackPrediction` uses score buffers that hold up to 25 scores for each type of violation. When a new observation comes in, it just pushes out the oldest one, so memory use per track stays constant, no matter how long the video runs. The `OCRStabilizer` only remembers the last seven OCR readings for each vehicle. After a license plate gets confirmed (when `lock_text` is set), the system skips OCR for that track in later frames. That way, it saves both CPU power and memory. Table II lists the major in-memory caches and their cleanup policies.

TABLE II. In-Memory Cache Inventory

Cache / Store	Data Type	Max Size	Cleanup Trigger
JOB_STORE	Job status, progress, errors	Per job	Manual / on completion
live_track_data	Per-track detection state	Active tracks only	Track age > 200 frames
TemporalDecisionFusion	EMA scores, history (deque 15)	Per track $\times$ 3 types	Every 50 frames
ViolationPredictor	Score history (deque 25 each)	Per track	Track age > 100 frames
OCRStabilizer	OCR readings (max 7 per vehicle)	Per active vehicle	When OCR confirmed
DatabaseGate_stored_today	Plate + type + date keys	Daily rolling	Pruned daily
prediction_cache	Latest predictor output	Per track	On cleanup cycle

B. Periodic Cleanup

Every 50 frames, the system runs a clean-up cycle. It goes through three methods: `pipeline.cleanup(frame_id)`, `ViolationPredictor.cleanup_old_tracks(frame_id)`, and `DatabaseGate.cleanup_old_tracks(frame_id)`. If a track hasn't shown up in the last 100 to 200 frames—the exact number depends on the module—it gets wiped from memory. This frees up space for vehicles that aren't on camera anymore.

C. Adaptive State Decay

When a vehicle disappears from view for a short time but is still being tracked, the TDF engine uses EMA decay to handle the skipped frames. Here's what happens: $\text{EMA}_v(t) \rightarrow \text{EMA}_v(t-1) \cdot \text{skip_decay}^{\Delta t}$,

where $\text{skip_decay} = 0.96$ and Δt is the number of skipped frames. A decay rate of 0.92 also helps to eliminate prejudice. This way, older EMA scores don't end up exaggerating the violation risk for vehicles that drop out of sight for just a moment.

D. Asynchronous Video I/O

The system loads annotated frames into an AsyncVideoWriter thread, which can hold up to 200 frames at a time. When the queue hits that limit, it means writing to disk can't keep up with inference—so the inference process pauses until there's room again. This setup keeps memory use in check, stopping it from ballooning as frames pile up, but still delivers solid throughput when you're using standard NVMe storage.

E. Persistence and Recovery

After 500 samples, the system saves adaptive threshold states in state/thresholds.json. Instead of logging each violation right away during inference, it saves them up and adds them to PostgreSQL all at once when it finishes processing a video. This way, there's less transaction overhead, and you don't end up with a messy violation log if something stops the process halfway.

VI. IMPLEMENTATION

A. Technology Stack

The backend runs on Python 3.10 or newer. FastAPI handles the REST endpoints, Uvicorn serves as the ASGI server, and SQLAlchemy takes care of the ORM. For async PostgreSQL connections, it's asyncpg. For model inference, the Ultralytics SDK covers all YOLO versions. EfficientNet-B0 gets fine-tuned with PyTorch, using torchvision transforms. DeepSORT tracking comes from deep-sort-realttime. EasyOCR pulls text from license plates. For video input and output, it's all OpenCV-Python.

B. Model Training Details

EfficientNet-B0 works as the HSRP classifier. Training happens in two phases: first, 5 epochs with the backbone frozen using ImageNet pre-trained weights; then, another 20 epochs for serious fine-tuning, following a cosine learning-rate schedule. The YOLOv10s plate detector trains for 100 epochs using 14,114 training and 2,048 validation images. For helmet detection, YOLOv8s uses a custom dataset with four classes: rider, face_no_helmet, face_helmet_good, and face_helmet_bad. To improve spatial grounding, a new class for number-plate localization was added. All models are exported to ONNX, so they're ready for production and can run on different devices.

C. Configuration and Deployment

Runtime settings live in a .env file: model paths, confidence thresholds, database URL, JWT secret, and frame_skip (set to 1 by default, so every frame gets processed). Authentication uses JWT with HS256, and access tokens last 30 minutes. The setup scales horizontally—spin up more stateless API workers behind a load balancer, all talking to the shared PostgreSQL database and a file-system mount for video files.

VII. EXPERIMENTAL RESULTS

Table I displays the performance metrics for each model. The HSRP classifier achieves 87.5% test accuracy with balanced precision and recall (0.88 for HSRP and 0.86 for NON_HSRP). This shows that EfficientNet-B0 performs effectively on the binary HSRP/NON_HSRP task, despite the significant visual resemblance of some plate variants.

TABLE I. Model Performance Summary

Model	Precision	Recall	F1-Score	mAP50
HSRP Classifier (EfficientNet-B0)	0.88	0.88 / 0.86	0.87 / 0.87	N/A
Plate Detector (YOLOv10s)	0.981	0.949	N/A	0.979
Helmet Detector (YOLOv8s)	0.78	0.888	N/A	0.896
Vehicle Detector (YOLOv11m)	0.91	0.87	N/A	0.94

Plate detection nails an mAP50 of 0.979 with YOLOv10s. That's mostly thanks to a huge training set and the fact that Indian license plates don't vary much in how they look. The helmet detector does a solid job picking up the rider and face_no_helmet classes (mAP50 over 0.739). But it struggles with face_helmet_bad—precision and recall drop to 0.373 and 0.283. Basically, the training data didn't have enough examples of helmets worn the wrong way. To work around this, TDF uses different confidence thresholds. Setting a lower threshold for helmet violations helps the system catch more of those bad-helmet cases, since the classifier tends to be extra careful there.

B. How Well Temporal Fusion Works

To see what temporal fusion actually brings to the table, we ran the system in two setups: single-frame mode (TDF off, just using a direct threshold), and the full TDF pipeline. We used a separate batch of dashcam footage, about 90 minutes long, for testing. The results? TDF cut false positives by 63.4%, and true positive recall only dropped by 4.8%. Precision jumped from 0.61 to 0.91 at the same settings. On top of that, the adaptive quality gate ($Q > 0.55$) filtered out 8.2% more of the TDF-confirmed events, mostly ones where the vehicle tracking got shaky because of frame-to-frame occlusion.

C. How Accurate Violation Prediction Gets

The VPE managed to send out early warnings for 81.4% of confirmed HSRP violations, usually about 7 frames before the actual infraction. For helmet violations, warning coverage hit 74.2%—not quite as high, probably because tracking rider posture and helmet position is trickier. Every violation report keeps a prediction_preceded flag, so we can dig deeper into how reliable these predictions are, which isn't an option with the other systems we looked at.

D. System Speed and Throughput

Running the pipeline on an NVIDIA RTX 3060 GPU with a 1080p video feed and frame_skip set to 1, we hit an average of 18.3 frames per second. That's compared to the incoming 25 fps, so we're running at about 1.37 times real time. If we

bump frame_skip up to 2, throughput jumps to 31.4 fps—now we’re at true real-time speed, though there’s a small tradeoff in fusion quality. From the moment a frame comes in to when it gets annotated, the average delay is just 54.7 milliseconds.

VIII. COMPARISON WITH EXISTING SYSTEMS

Table III includes a detailed comparison of the proposed system to commercial Indian traffic enforcement platforms as well as the two most relevant academic systems. The primary distinctions include temporal fusion, multi-layer false-positive filtering, adaptive thresholds, and prediction-aware storage, which were not present in previous systems.

TABLE III. Comparative Analysis

Feature	Existing Systems (EFKON/AIRPIX)	UrbanEye [1]	UrbanFlow [2]	Smart HSRP (Proposed)
Temporal Fusion	X	X	X	✓
Multi-Layer Filtering	X	Partial	X	✓ (7-Layer)
Adaptive Thresholds	X	X	X	✓
HSRP Verification	X	X	X	✓
Helmet Detection	Limited	✓	X	✓
OCR Stabilization	X	Partial	✓	✓
Violation Prediction	X	X	X	✓
Human Review Needed	Yes	Sometimes	Yes	Minimized
Specialized Hardware	Required	Optional	Optional	Optional

Most ANPR-based systems have a big flaw—they judge violations by looking at just one frame. If a car runs a red light in that exact moment, great, the system catches it. But this approach falls apart with more complex or ongoing violations, like missing HSRP plates or riders not wearing helmets. Sometimes, a vehicle only appears partially in a few frames before you finally get a clear shot, and that’s a problem the single-frame logic just can’t handle.

UrbanEye [1] and UrbanFlow [2] both made real academic progress, but they still only work on single frames. UrbanEye nails detection accuracy in controlled tests, but doesn’t say much about how often it makes mistakes out in the real world. UrbanFlow is built to scale, thanks to its microservice setup, but it still can’t reliably track things over time. That’s where our approach steps in—using a solid probabilistic model that builds up evidence across multiple frames, powered by the EMA-driven TDF framework.

IX. CONCLUSION AND FUTURE WORK

Here, we introduced the Smart HSRP Monitoring System—a platform that actually pinpoints traffic violations automatically and accurately. We brought four key pieces to the table: Temporal Decision Fusion, Adaptive Threshold Learning, Database Gating, and a proactive Violation Prediction Engine. With these, we cut down false positives by 63.4% compared to single-frame systems, all while hanging onto a 95.2% recall rate. The system runs on regular GPUs, connects smoothly to existing PostgreSQL-based enforcement setups, and leaves a clear evidence trail. Every violation comes with a quality score, prediction flags, and marks for manual review.

Looking ahead, we’re focusing on three things. First, making edge deployment easier by using model quantization (INT8) and TensorRT serialization. That way, we can run things in real time on NVIDIA Jetson devices, no need for heavy-duty data center GPUs. Second, expanding the system to catch more violations like triple riding, not wearing seatbelts, and driving on the wrong side. This means building logic to track multiple people and gathering more training data. Third, we’ll use federated learning so each camera can update its thresholds on its own, without sending raw video to a central server—solving some of the big privacy concerns in public spaces.

The system’s modular design means it works with all sorts of devices. Our approach—gathering solid temporal evidence—lays the groundwork for the next wave of automated traffic enforcement, in India and anywhere else that needs it.

REFERENCES

- [1] S. Gowroju, S. Choudhary, N. Sathwik, A. Banu, E. V. Kumar and N. Ashritha, "UrbanEye: Deep Learning Enhanced Two-Wheeler Traffic Rule Violation Detection using YOLO and OCR," 2025 International Conference on Innovations in Intelligent Systems: Advancements in Computing, Communication, and Cybersecurity (ISAC3), Bhubaneswar, India, 2025, pp. 1-5, doi: 10.1109/ISAC364032.2025.11156855.
- [2] S. Patil, O. Baji, M. Bagul, S. Bhangare and K. Bhutada, "Urban Flow - A Integrated Smart Traffic Management System," 2025 5th Asian Conference on Innovation in Technology (ASIANCON), PIMPRI, India, 2025, pp. 1-7, doi: 10.1109/ASIANCON66527.2025.11281127.
- [3] Y. Fang, R. Zhang, Q. F. Wang and K. Huang, "Action Recognition in Videos with Temporal Segments Fusions," in Advances in Brain Inspired Cognitive Systems (BICS 2019), Lecture Notes in Computer Science, vol. 11691, Springer, Cham, 2020, doi: 10.1007/978-3-030-39431-8_23.
- [4] G. Jocher et al., "Ultralytics YOLO," Version 8.0.0, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [5] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in Proc. 36th Int. Conf. Machine Learning (ICML), PMLR, vol. 97, pp. 6105-6114, 2019.
- [6] N. Wojke, A. Bewley and D. Paulus, "Simple Online and Realtime Tracking with a Deep Association Metric," in Proc. IEEE Int. Conf. Image